

Project 1: Native PDF Chunker + RAG Question Answering

Objective

Build a tool that ingests a PDF, splits it intelligently into chunks, stores them for retrieval, and answers questions based only on the PDF content.

Requirements

Upload a PDF.

Chunk the PDF by semantic boundaries (paragraphs, headings, list items) – not just fixed page breaks.

Generate a searchable representation for each chunk.

Accept a natural language question, retrieve the most relevant chunks, and use them to generate an answer.

Work with local models only.

Constraints

No paid APIs for embeddings or LLM.

UI: simple interface (file upload, question box, answer + citations).

Evaluation Criteria

Faithfulness: answer is fully grounded in the PDF (no hallucination).

Citation quality: chunk boundaries affect retrieval relevance.

Speed: chunking a 50-page PDF under 1 minute.

Helpful Hints

Split by headings, paragraph breaks, and sentence endings.

Use overlapping chunks to preserve context across boundaries.

For retrieval, compare the question against chunk representations and take the top-k.

Instruct the LLM to quote the source chunk when answering.

Project 2: Prompt-Based SQL Searching (Text-to-SQL)

Objective

Convert English questions into SQL queries, run them on a database, and return results in plain English.

Requirements

Pre-load a SQLite database (choose a domain: sales, library, employees, etc.).

User types a question → system generates SQL, executes it automatically.

If SQL fails, the system self-corrects (re-prompt with error message) up to 2 times.

Display: the generated SQL, query results as a table, and a one-sentence English summary.

Constraints

Free LLM API or local model only – no fine-tuning, pure prompt engineering.

UI: three sections – input, SQL, results table, report (optional).

Evaluation Criteria

Accuracy on complex queries (JOINS, GROUP BY, WHERE).

Error recovery: can it fix its own wrong SQL after seeing the error?

Prompt design: clarity of schema description and few-shot examples.

Helpful Hints

Provide the LLM with the database schema (tables and columns) as context.

Ask it to first explain the SQL in words, then generate it.

When SQL errors occur, feed the exact error message back with “correct the query”.

Cache repeated questions to save API calls.

Project 3: Prompt Enhancer for Image Generation

Objective

Build a tool that takes a rough user prompt (e.g., “big red building”) and outputs an optimised, detailed prompt suitable for image generation models (like Midjourney, DALL-E, or Stable Diffusion). No actual image generation required – only the enhanced prompt.

Requirements

Input: short, vague, or poorly structured prompt (1–5 words or a simple phrase).

Output: a rich, descriptive prompt (50–200 words) that includes:

Subject details (colour, texture, material)

Lighting, atmosphere, mood

Composition, camera angle, lens type

Style references (e.g., “cinematic”, “watercolour”, “cyberpunk”)

Provide three alternative enhanced prompts for the same input.

Include a short “reasoning” section explaining why each enhancement improves the original.

Constraints

Free LLM API or local model only (no image generation API needed).

UI: input box, generate button, three output cards with the enhanced prompts + reasoning.

Must handle diverse domains (landscape, portrait, object, fantasy, realistic).

Evaluation Criteria

Prompt quality: does it add specific, actionable details without distorting the original idea?

Creativity: how different are the three alternatives (e.g., different lighting or styles)?

Transparency: the reasoning should teach the user about prompt engineering.

Helpful Hints

Use a system prompt like: "You are a prompt engineer for image generation. Expand the user's short idea into a detailed prompt. Include: subject, environment, lighting, composition, mood, style, and technical camera details."

Ask the LLM to brainstorm multiple angles – e.g., "give me three versions: one photorealistic, one artistic, one surreal."

The reasoning can be a simple bullet list under each prompt.

No need to call any image API – just text in, text out.

Project 4: Voice-Based AI Assistant

Objective

Create a voice assistant that listens to a spoken question, processes it with an LLM, and responds with spoken audio.

Requirements

Record audio via microphone (button push to start/stop – no wake word).

Transcribe speech to text (free ASR API or local model).

Send text to an LLM for a helpful answer (general knowledge or a narrow domain you choose).

Convert LLM's text response to speech (free TTS API or local model).

Play the audio response in the browser/app.

Constraints

All components (STT, LLM, TTS) must use free tiers or open-source local models.

UI: record button, transcript area, play button for response.

Latency <5 seconds for a short question.

Evaluation Criteria

Transcription accuracy (accent, background noise).

Response quality: relevant and conversational.

TTS naturalness (not overly robotic).
End-to-end speed.

Helpful Hints

Use realtime api or voice apis

Project 5: Train a Model with Sample Dataset – Min 95% Accuracy

Objective

Train a machine learning model on a real dataset to achieve at least 95% accuracy on a test set.

Requirements

Choose a classification or regression task (spam detection, species classification, price prediction, etc.).

Use a public dataset or collect your own (e.g., 20 images per class).

Split: 80% train, 20% test (stratified).

No prompting of large pre-trained models (LLMs). Traditional ML (random forest, gradient boosting, small neural nets) or fine-tuning a small transformer (e.g., DistilBERT) is fine.

Achieve $\geq 95\%$ test accuracy. If impossible due to dataset difficulty, justify and show improvement over a simple baseline.

Constraints

No paid compute – free Google Colab or local CPU/GPU.

Provide: code, test accuracy, confusion matrix, and a 1-page report explaining feature engineering and hyperparameter choices.

Evaluation Criteria

Accuracy ($\geq 95\%$ = full marks).

Generalisation (train vs test gap $< 5\%$).

Reproducibility.

Quality of explanation (why that model? which features mattered?).

Helpful Hints

Start with a simple baseline (e.g., logistic regression) to know the lower bound.

Engineer features – combine, bin, create interaction terms.

Use cross-validation instead of a single validation split for small datasets.

If accuracy plateaus, try ensembles (voting classifier) or a small neural net with dropout.

Avoid data leakage – scale after splitting.